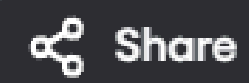


QUESTION:14 CONSTRUCT A TREE TRAVERSAL

main.c	Output
<pre>1 #include <stdio.h> 2 #include <stdlib.h> 3 struct Node { 4 int data; 5 struct Node *left; 6 struct Node *right; 7 }; 8 struct Node* createNode(int data) { 9 struct Node* node = (struct Node*)malloc(sizeof(struct Node)); 10 node->data = data; 11 node->left = NULL; 12 node->right = NULL; 13 return node; 14 } 15 void preorder(struct Node* root) { 16 if (root == NULL) 17 return; 18 printf("%d ", root->data); 19 preorder(root->left); 20 preorder(root->right); 21 } 22 void inorder(struct Node* root) { 23 if (root == NULL) 24 return; 25 inorder(root->left); 26 printf("%d ", root->data); 27 inorder(root->right); 28 } 29 void postorder(struct Node* root) {</pre>	<pre>Preorder Traversal: 1 2 4 5 3 6 7 Inorder Traversal: 4 2 5 1 6 3 7 Postorder Traversal: 4 5 2 6 7 3 1 === Code Execution Successful ===</pre>

main.c



Share

R

```
22 void inorder(struct Node* root) {
23     if (root == NULL)
24         return;
25     inorder(root->left);
26     printf("%d ", root->data);
27     inorder(root->right);
28 }
29 void postorder(struct Node* root) {
30     if (root == NULL)
31         return;
32     postorder(root->left);
33     postorder(root->right);
34     printf("%d ", root->data);
35 }
36 int main() {
37     struct Node* root = createNode(1);
38     root->left = createNode(2);
39     root->right = createNode(3);
40     root->left->left = createNode(4);
41     root->left->right = createNode(5);
42     root->right->left = createNode(6);
43     root->right->right = createNode(7);
44     printf("Preorder Traversal: ");
45     preorder(root);
46     printf("\nInorder Traversal: ");
47     inorder(root);
48     printf("\nPostorder Traversal: ");
49     postorder(root);
50     return 0;
51 }
```